



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Modelado de un CPS mediante orientación a agentes y DSL

Actividad Semana 3 — caso SVIF: modelo iStar 2.0 (vistas SD y SR/híbrida),
su representación equivalente en el DSL para CPS y la correspondencia entre ambos.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

¿Por qué orientación a agentes para SVIF?

SVIF involucra componentes de software y hardware que interactúan de manera distribuida y persiguen objetivos que pueden entrar en conflicto:

PRIVACIDAD DE DATOS

VS.

OPORTUNIDAD DE LA IDENTIFICACIÓN

SEGURIDAD DEL RECINTO

VS.

EFICIENCIA DE RECURSOS

La **orientación a agentes** permite capturar objetivos, responsabilidades y dependencias desde etapas tempranas (Cares, Sepúlveda & Navarro, 2019).

iStar 2.0 (Dalpiaz, Franch & Horkoff, 2016) provee los constructos necesarios: actores, agentes y roles; objetivos, tareas, recursos y cualidades; y enlaces de dependency, refinement, needed-by, qualification y contribution.

ENTREGABLE

Modelo CIM en diagrams.net

Archivo cim-istar-svif.drawio con dos páginas:

VISTA SD

VISTA SR/HÍBRIDA

Biblioteca: scratchpad iStar 2.0 (../recursos-comunes/librerias-drawio/).

27 elementos inventariados (3 actores · 2 goals · 10 tasks · 5 resources · 4 qualities · 3 dependums).

Identificación de actores

ACTOR	TIPO ISTAR	JUSTIFICACIÓN
Face Monitor Component	AGENTE	Entidad concreta (nodo ESP32-CAM) con autonomía para percibir y decidir.
Access Actuator Component	AGENTE	Entidad concreta (nodo ESP32) que actúa sobre el mundo físico.
Administrador de Seguridad	ACTOR	Interesado humano; depende del sistema para mantener el recinto controlado.

Decisión de modelado: los nodos ciberfísicos se modelan como **agentes** (instancias concretas con autonomía) y al humano como **actor** genérico, siguiendo la distinción de iStar 2.0.

Dependency & Refinement

PASO 1 · DEPENDENCY (VISTA SD)

Tres dependencias clave

- El **Administrador de Seguridad** depende del **Access Actuator** para el goal «Acceso controlado al recinto» y el recurso «Registro de accesos».
- El **Access Actuator** depende del **Face Monitor** para el recurso «Evento de identificación»: no puede decidir sobre el acceso sin que alguien perciba e identifique a la persona.

PASO 2 · REFINEMENT (VISTA SR)

AND en el monitor · AND + OR en el actuador

Monitor · «Monitorear presencia de personas» se refina **AND** en:

- Capturar imagen → Detectar rostro → Identificar rostro → Publicar evento
- «Identificar rostro» se refina en «Comparar con rostros enrolados»

Actuador · «Controlar acceso al recinto» se refina **AND** en:

- Evaluar autorización · Gestionar respuesta · Registrar evento
- «Gestionar respuesta» se refina **OR** en: **Desbloquear** v **Activar alarma**

NeededBy · Qualification · Contribution

PASO 3 · NEEDED BY

Recursos por tarea

- Capturar imagen → **Sensor cámara OV2640**
- Comparar rostros → **Base de rostros enrolados**
- Desbloquear cerradura → **Cerradura electromecánica**
- Activar alarma → **Alarma sonora**
- Registrar evento → **Bitácora de accesos**

PASO 4 · QUALIFICATION

Cualidades

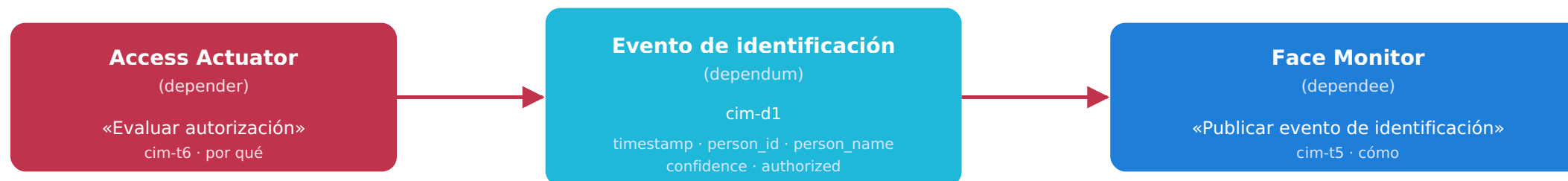
- «**Identificación oportuna**» califica a «Monitorear presencia de personas»
- «**Oportunidad de la respuesta**» califica a «Controlar acceso al recinto»

PASO 5 · CONTRIBUTION

Aportes a softgoals

- **Comparar con rostros enrolados** → **help**
«Privacidad de datos personales»
- **Publicar evento de identificación** → **hurt**
«Privacidad» (transmite datos minimizados)
- **Registrar evento de acceso** → **help** «Trazabilidad de accesos»

La dependencia que articula SVIF



Esta dependencia recorre toda la cadena: del CIM al dependum PIM, al hilo MQTT, al struct PSM y al callback en el actuador.

La dependencia `cim-d1` (Evento de identificación) es el **eje** que recorre toda la cadena MDD: CIM → dependum del PIM → hilo MQTT → struct PSM → callback en el actuador.

Inventario de elementos (cim-*)

Monitor (cim-a1)

ID	ELEMENTO	TIPO
cim-g1	Monitorear presencia de personas	goal
cim-t1..t5	Capturar / Detectar / Identificar / Comparar / Publicar	task
cim-r1	Sensor de cámara OV2640	resource HW
cim-r2	Base de rostros enrolados	resource SW
cim-q1	Identificación oportuna	quality
cim-q2	Privacidad de datos personales	quality

Administrador (cim-a3) + dependums cim-d1, cim-d2, cim-d3.

Actuador (cim-a2)

ID	ELEMENTO	TIPO
cim-g2	Controlar acceso al recinto	goal
cim-t6..t10	Evaluar / Gestionar respuesta / Desbloquear / Alarmar / Registrar	task
cim-r3	Cerradura electromecánica	resource HW
cim-r4	Alarma sonora	resource HW
cim-r5	Bitácora de accesos	resource SW
cim-q3	Oportunidad de la respuesta	quality
cim-q4	Trazabilidad de accesos	quality

Lo que el modelo asume (y deja abierto)

ASUNCIONES TOMADAS

Cerradas en el CIM

- Los nodos ciberfísicos son **agentes**; el humano es **actor**.
- El **OR** en la respuesta de acceso captura una decisión de diseño **abierta a nivel CIM**: el modelo no prescribe cuándo desbloquear o alarmar.
- Las cualidades se limitan a las **4 con influencia arquitectónica demostrable** (oportunidad ×2, privacidad, trazabilidad).

LO QUE SE DIFIERE

Queda para fases siguientes

- **Lógica del OR**: cuándo desbloquear o alarmar (fase Code: política de evaluarAutorizacion).
- Otras cualidades (eficiencia energética, confiabilidad) → requisitos en diseno-del-sistema.md.
- Trazabilidad: IDs cim-* se conservan en el PIM mediante id_cim_parent (Semana 4).

El DSL para CPS: constructos más cercanos a la implementación

El modelo AO captura **objetivos, responsabilidades y dependencias**, pero una plataforma concreta necesita **hilos, funciones, temporizadores, variables de estado y acceso a recursos físicos**.

El DSL es el **punto**: conserva las decisiones del modelo AO, pero las expresa con conceptos fáciles de mapear al software, de forma **independiente de tecnología**.

Archivo entregado: `pim-dsl-svif.drawio`, construido con la biblioteca *scratchpad PIM-DSL* de `diagrams.net`.

CONSTRUCTOS DEL DSL USADOS EN SVIF

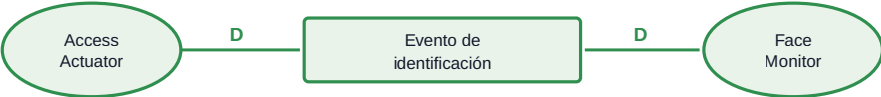
CONSTRUCTO	ROL
CP Component	Contenedor de un nodo CPS
On Interval Action	Proceso periódico (<code>interval_in_milliseconds</code>)
On Demand Action	Procedimiento activado por evento
HW Resource	Sensor / actuador / E-S
SW Resource	Datos (<code>data_structure</code>)
AND / OR	Operadores de refinamiento
Message Sender / Receiver	Comunicación entre CPC

Cómo se representó cada elemento del modelo iStar

ISTAR 2.0	DSL	POR QUÉ
Agent	CP Component	Nodo computacional
Goal	On Interval Action	Objetivo persistente → evaluación continua
Task	On Demand Action	Procedimiento concreto
Resource físico	HW Resource	Elemento del mundo físico
Resource informacional	SW Resource	Estructura de datos
AND / OR	Operadores AND / OR	Se conservan explícitos
NeededBy	From-To Relation	Acción → recurso requerido
Dependency	Sender + Receiver	Delegación → mensaje en red
Quality	qualification_array contribution_array	Sin símbolo propio: atributo

LA DEPENDENCIA CENTRAL, TRADUCIDA

CIM (iStar)



se traduce en

PIM (DSL)



El *dependum* pasa a ser el *dependum_data_structure* del mensaje: timestamp, person_id, person_name, confidence, authorized.

Qué se gana y qué se pierde al pasar de AO a DSL

SE GANA

Información que el DSL exige

- **Períodos:** `interval_in_milliseconds` — 500 ms percepción, 250 ms actuación.
- **Estructura de los datos:** `data_structure` de los recursos SW y `dependum_data_structure` del mensaje.
- **Parámetros de entrada/salida** de cada acción.

Decisiones que el modelo AO deliberadamente **no** fijaba, y que aquí hay que tomar.

SE PIERDE

Expresividad que el DSL no dibuja

- **Los softgoals dejan de ser nodos:** la contribución *hurt* de «Publicar evento» sobre la privacidad pasa de arista visible a atributo que hay que ir a leer.
- **Actor / rol / agente** colapsan en CP Component.
- El **actor humano** (Administrador) queda fuera: el DSL modela nodos computacionales.
- El **criterio del OR** no se expresa: se reintroduce en el código.

No es un defecto del DSL: cada nivel retiene lo que necesita para la transformación siguiente. *id_cim_parent* es el mecanismo que permite volver al CIM a recuperar el «porqué».

Qué entrega Semana 3

- ✓ **Modelo iStar** (`cim-istar-svif.drawio`): vista SD + vista híbrida SD/SR, con Goals, Softgoals, Tasks y Resources, y la dependencia «Evento de identificación» entre los dos CPC.
- ✓ **Modelo DSL** (`pim-dsl-svif.drawio`): representación equivalente con la biblioteca *scratchpad PIM-DSL*.
- ✓ **Correspondencia AO-DSL** explicada elemento por elemento, con el análisis de lo que se gana y lo que se pierde en la traducción.
- ✓ Trazabilidad `cim-*` → `id_cim_parent`, base del proceso MDD4CPS de la Semana 4.

REFERENCIAS

Cares, C., Sepúlveda, S., & Navarro, C. (2019). *AGENT-ORIENTED ENGINEERING FOR CYBER-PHYSICAL SYSTEMS*. ICITS 2019. Springer.

Dalpiaz, F., Franch, X., & Horkoff, J. (2016). *ISTAR 2.0 LANGUAGE GUIDE*. *arXiv:1605.07767*.

Navarro, C., Devia, L., Labra Gayo, J. E., & Cares, C. (2025). *AN AGENT-ORIENTED MODEL-DRIVEN DEVELOPMENT PROCESS FOR CPS*. CIBSE 2025. SBC.

PRÓXIMA PARADA

Semana 4 • MDD4CPS

Transformar este PIM en PSM (C++ Arduino) → Code, y medir cuánto del código se genera automáticamente.